Project 2: Modelling daily cycling demand in Edinburgh

Group members: Alice Miller, Saba Sitter, Jake Saunders

Word count: [XXX words]

1 Introduction

The "cycle daily" dataset consists of approximately 2,180 daily observations ranging from 2020 to 2025, sourced from Edinburgh's network of automatic cycling counters, along with local weather and temporal data. This data comes from sensors at specific locations across the city, so it's possible not every cyclist is accounted for. However, they are a good indicator of how cycling demand varies across different days and weather patterns. Edinburgh relies heavily on public transport for its residents and tourists to navigate the city. For future funding allocation and planning, it is important to know how many cyclists are on the road in order to evaluate whether the infrastructure (cycle lanes, crossings etc) and investment in schemes (rentable Voi bikes) need to adapt to meet demand. The modelling goal is to develop a sequence of increasingly complex linear regression models to isolate how temporal and weather variables, such as the day of the week and the temperature, influence cycling demand. Ultimately, the goal is to extrapolate cycling demand to the future so that Edinburgh City Council and Transport Scotland can optimise infrastructure investment over the next decade to align with their active travel priorities.

2 Exploratory data analysis

To begin investigating the impact of weather on the number of daily cyclists, we begun with some exploratory data analysis. Table 1 displays the summary statistics for the weather data and number of daily cyclists is as follows:

2.1 Summary Statistics table for count, temp_mean, temp_min, temp_max

Table 1: Summary Statistics for count, temp mean, temp min, temp max

Variable	Mean	SD	Min	Max
count	10815.47	4728.88	1264.00	43864.00
temp_mean	9.69	4.79	-8.08	22.96
temp_min	5.92	4.98	-14.00	18.00
temp_max	13.15	5.31	-3.00	31.00

We can see that the number of daily cyclists varies from $1,264$ to $43,864$, an increase of $\approx 3470\%$, and the variation is very high. Intuitively, one would think that a warmer average weather would lead to more cyclists.

We then plotted a time series of the daily cyclist count from 2020 to 2025.

2.2 Time series of daily counts 2020-2025 with a smoother.

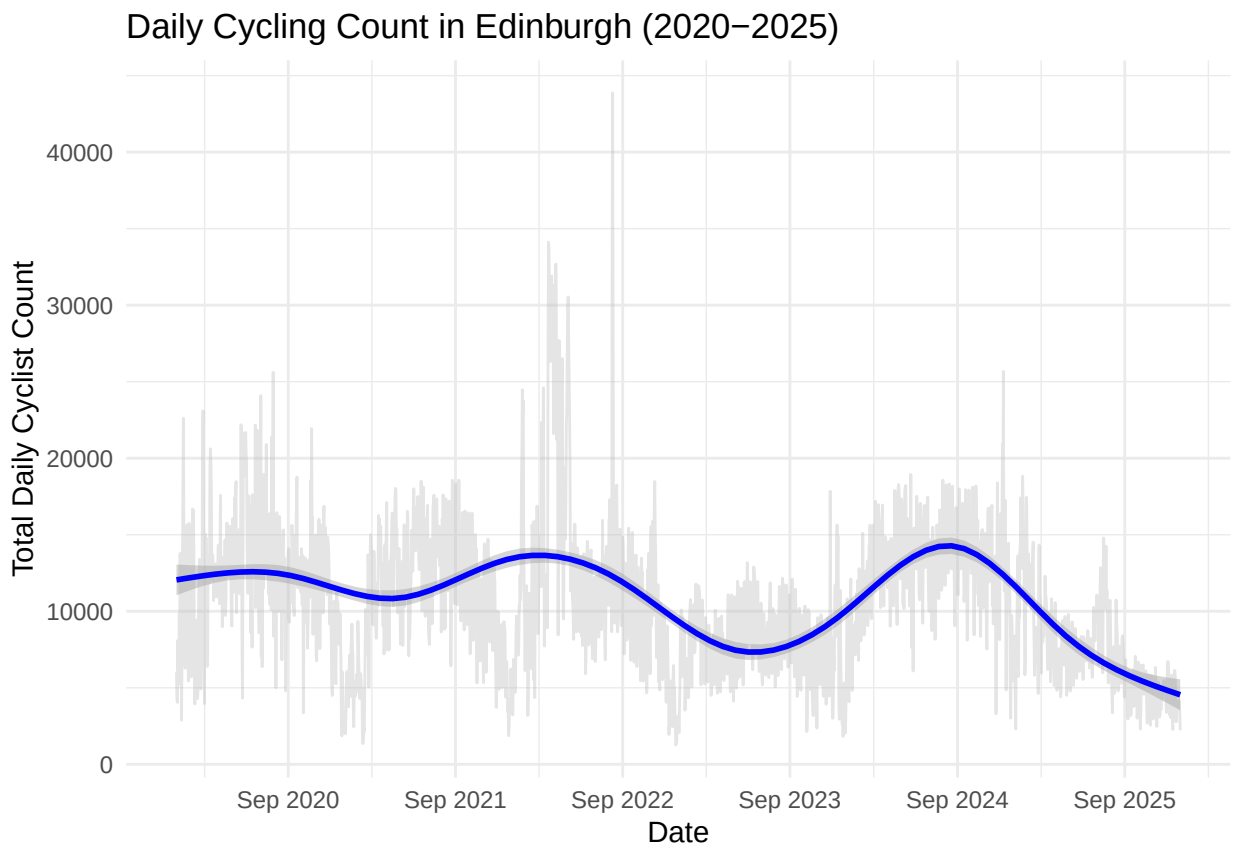

Figure 1: Time series of daily counts 2020-2025 with a smoother

We can see from figure 1 that there are yearly fluctuations, with peaks and troughs occuring yearly. Overall, the cyclist count seems to be decreasing. There is a visible peak in 2022, which we investigate. We then created boxplots of the daily cyclist count by month and by the day of week.

2.3 Boxplots of count by month and by day of week.

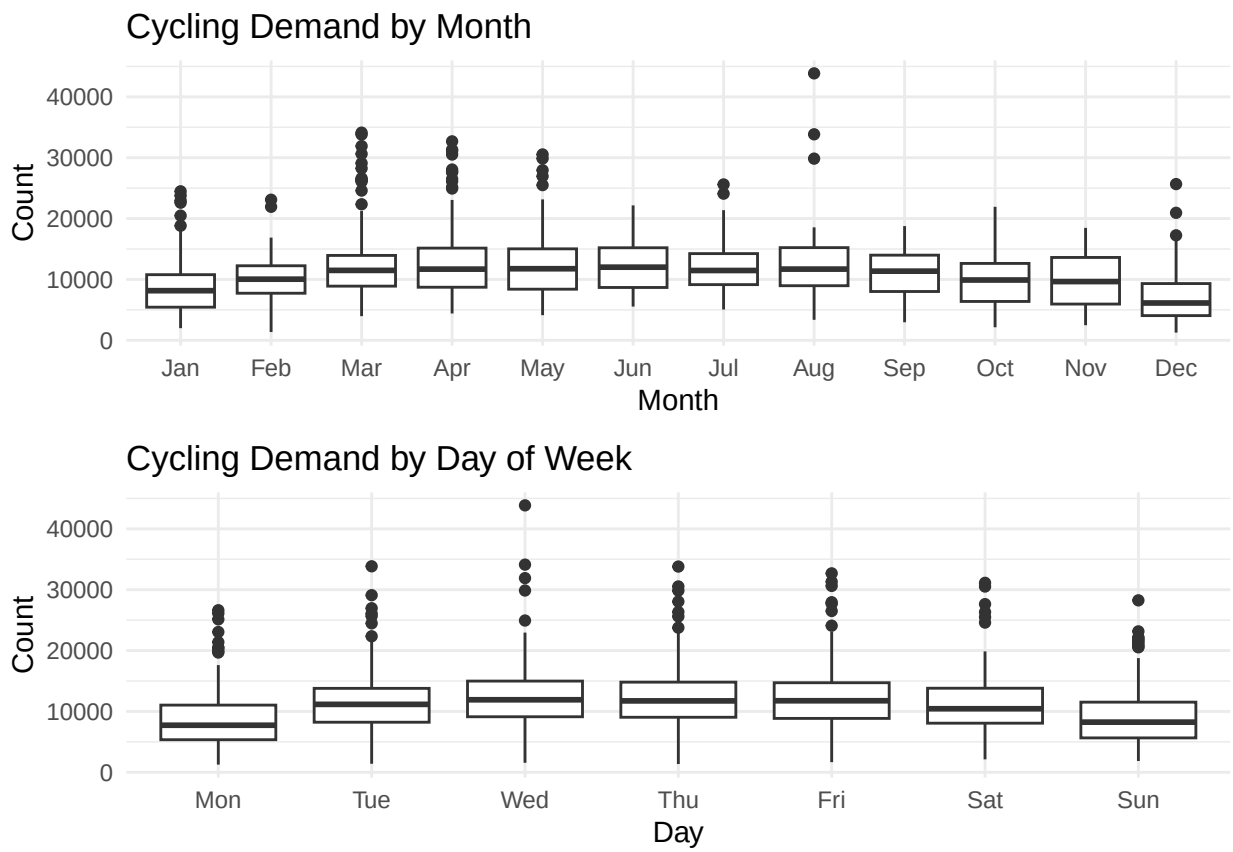

Figure 2: Boxplots of count by month and by day of week

We can see from figure 2 that the winter months have slightly lower cyclist counts, with the other months remaining relatively constant. This indicates that the month does not have a tremendous impact on the number of daily cyclists outside of January and December. From the second boxplot, there appear to be less cyclists on Mondays and Sundays, with the other days again appearing relatively constant.

2.4 Scatter plot of count vs temp_mean.

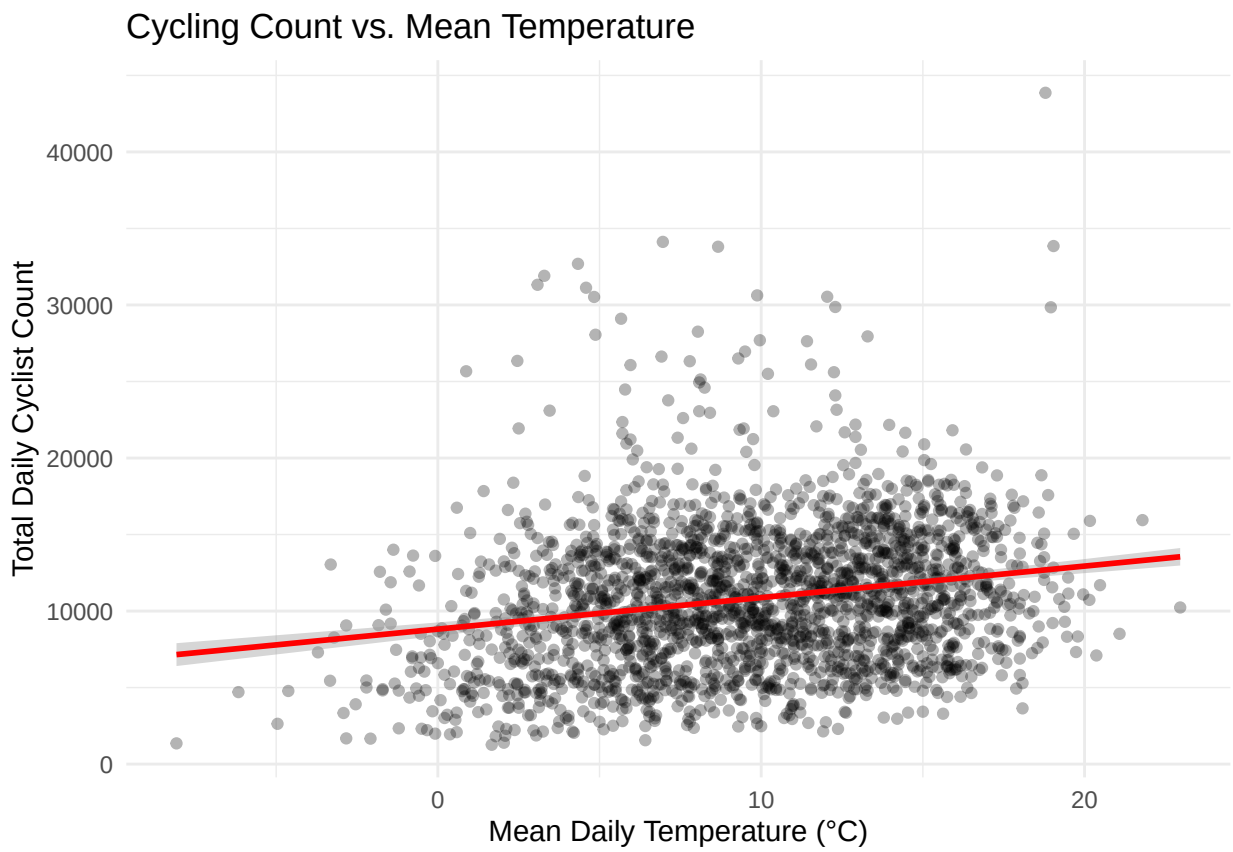

Figure 3: Scatter plot of count vs temperature mean

Figure 3 shows a scatter plot of daily counts vs the mean temperature for that day, with the red line being the regression line. There is a general trend as the temperature increases, so does the daily count which is as expected.

2.5 Extra Plots

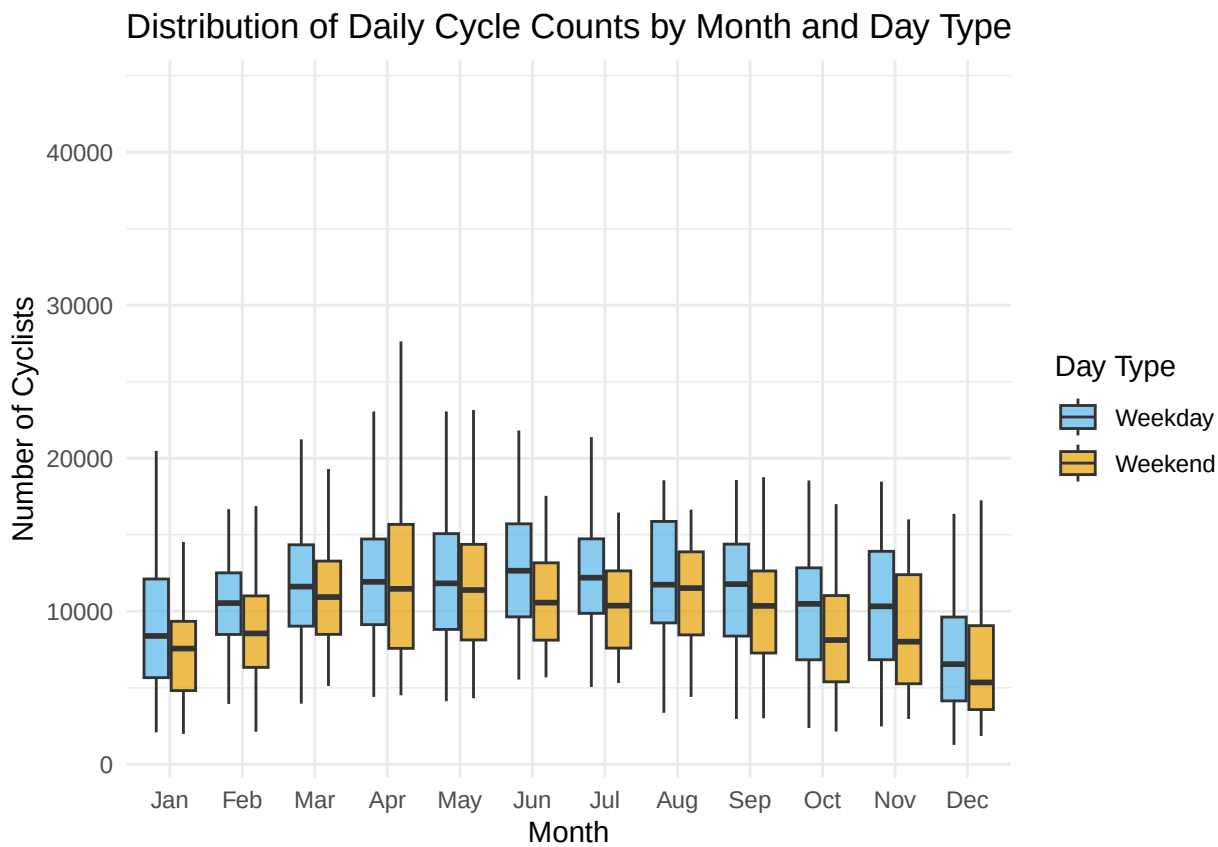

Figure 4: Boxplots of count vs weekend or weekday over the months (across 2020 to 2025)

Figure 4 shows for each month a boxplot for the different day types (weekends or weekdays). Generally the medians tend to be higher for the weekdays across all the months, but by varying amounts. June and Novemeber seems to have the biggest variance between the two days types

3 Baseline model M0

The equation for M0 is:

$$\text{count}_i = \beta_0 + \beta_1 \text{temp_mean}_i + \beta_2 \text{weekend}_i + \beta_3 \text{month}_i + \epsilon_i$$
$$\epsilon_i \sim N(0, \sigma^2)$$

Table 2 shows the summary statistics for model 0.

Table 2: Summary of model M)

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	10385.82	267.02	38.90	0
temp_mean	247.18	20.77	11.90	0

	Estimate	Std. Error	t value	Pr(>|t|)
as.numeric(weekend)	-1246.89	214.17	-5.82	0
as.numeric(month)	-247.18	28.81	-8.58	0

4 Proposed models M1–M3

4.1 Model M1: Trend and factor

4.2 Model M2: Nonlinear temperature

4.3 Model M3: [your extension title]

5 Evaluation

5.1 Statistical Performance

5.2 Practical Performance

6 Discussion and limitations

7 Appendix: Code

All analysis code is in `code.R` and is reproduced below.

```
# ============================================================
# Project 2: Modelling daily cycling demand in Edinburgh
# Authors: Alice Miller, Saba Sitter, Jake Saunders
# ============================================================

#
# Place the code needed in the Report_project02.Rmd, including documentation.
#
# 0. Packages
library(ggplot2); library(dplyr); library(tidyr)
library(lubridate); library(knitr); library(kableExtra)
library(patchwork); library(broom); library(gridExtra)

# 1. Load data
load('cycle_daily_df.Rdata')

# 2.1 Data Wrangling

cycle_daily_df <- cycle_daily_df %>%
  mutate(
    # Task 1: month as ordered factor for plots/inference
    month = factor(month,
                   levels = 1:12,
                   labels = month.abb,
```

```r
                    ordered = TRUE),

    # Task 2: dow with explicit levels
    dow = factor(dow,
                 levels = c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"),
                 ordered = TRUE),

    # Task 3: trend as integer days since 2020-01-01
    trend = as.numeric(date - date[1], units = "days")
    )

# 2.2 exploratory plots

# Summary table for count, temp_mean, temp_min, temp_max
summary_stats <- cycle_daily_df %>%
  summarise(across(c(count, temp_mean, temp_min, temp_max),
                   list(Mean = ~mean(.x),
                        SD = ~sd(.x),
                        Min = ~min(.x),
                        Max = ~max(.x)),
                   .names = "{.col}.{.fn}")) %>% # use dots in variable names instead of underscore
  # Reshape for a vertical table
  pivot_longer(everything(),
               names_to = c("Variable", "Statistic"),
               names_sep = "\\.") %>% # Split at the dot instead of the underscore
  pivot_wider(names_from = Statistic, values_from = value)


# Time series of daily counts 2020-2025 with a smoother
plot_timeseries <- ggplot(cycle_daily_df, aes(x = date, y = count)) +
  geom_line(alpha = 0.4, color = "gray") +
  geom_smooth(method = "gam", color = "blue", se = TRUE) + # The smoother line
  scale_x_date(date_breaks = "12 months", date_labels = "%b %Y") +
  labs(
    title = "Daily Cycling Count in Edinburgh (2020-2025)",
    x = "Date",
    y = "Total Daily Cyclist Count"
  ) +
  theme_minimal()

# Boxplots of count by month and by day of week.

# Boxplot by Month
plot_month <- ggplot(cycle_daily_df, aes(x = month, y = count)) + # Use fill = month to add colour
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Cycling Demand by Month", x = "Month", y = "Count") +
  theme_minimal()

# Boxplot by Day of Week
plot_dow <- ggplot(cycle_daily_df, aes(x = dow, y = count)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Cycling Demand by Day of Week", x = "Day", y = "Count") +
  theme_minimal()
```

```r
# Scatter plot of count vs mean temperature
plot_temp_scatter <- ggplot(cycle_daily_df, aes(x = temp_mean, y = count)) +
  geom_point(alpha = 0.3) + # alpha changes the opaqueness of the points
  geom_smooth(method = "lm", color = "red", se = TRUE) +
  labs(
    title = "Cycling Count vs. Mean Temperature",
    x = "Mean Daily Temperature (°C)",
    y = "Total Daily Cyclist Count"
  ) +
  theme_minimal()

##non trivial plot
plot_weekend_seas <- ggplot(cycle_daily_df, aes(x = month, y = count, fill = factor(weekend))) +
  geom_boxplot(outlier.shape = NA, alpha = 0.7) + # Clean boxes for the "average"
  theme_minimal() +
  scale_fill_manual(values = c("0" = "#56B4E9", "1" = "#E69F00"),
                    labels = c("Weekday", "Weekend"), name = "Day Type") +
  labs(title = "Distribution of Daily Cycle Counts by Month and Day Type",
       x = "Month",
       y = "Number of Cyclists")


# 3. Model Fitting

# set the seed, defined test and training data,
n = 2192
set.seed(1234) # for reproducibility
idx <- sample(1:n, size = round(0.7*n))
train <- cycle_daily_df[idx, ]
test  <- cycle_daily_df[-idx, ]

# Note: Use factor(month) in formulas for M1-M3
m0 <- lm(count ~ temp_mean + as.numeric(weekend) + as.numeric(month), data = cycle_daily_df)
## m0 + factor(month) unsure what double parameters - choose factor version to avoid collinearity
m1 <- lm(count ~ temp_mean + as.numeric(weekend) + trend +
           factor(month) + factor(dow), data = cycle_daily_df)
m2 <- lm(count ~ I(temp_mean^2) + as.numeric(weekend) + trend +
           factor(month) + factor(dow), data = cycle_daily_df )
#m3 <- lm()



#diagnostic plots for first 3 models
#
#  Residuals vs fitted and normal QQ diagnostic plots, side by side.
#  Brief assessment: what does M0 get right and wrong?
#  What motivates M1?

#m0

# Add residuals and fitted values to the dataframe.
m0_diag <- augment(m0)
```

```r
# Plot Residuals vs Fitted for m0
p1_m0 <- ggplot(m0_diag, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(method = "loess", se = FALSE, color = "blue") +
  labs(title = "Residuals vs Fitted",
       x = "Fitted Values",
       y = "Residuals") +
  theme_minimal()

# Plot Normal Q-Q for m0
p2_m0 <- ggplot(m0_diag, aes(sample = .resid)) +
  stat_qq(alpha = 0.5) +
  stat_qq_line(color = "red") +
  labs(title = "Normal Q-Q Plot",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()

# m1
# Add residuals and fitted values to the dataframe.
m1_diag <- augment(m1)

# Residuals vs Fitted for M1
p1_m1 <- ggplot(m1_diag, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(method = "loess", se = FALSE, color = "blue") +
  labs(title = "M1: Residuals vs Fitted",
       x = "Fitted Values",
       y = "Residuals") +
  theme_minimal()

# Normal Q-Q Plot for M1
p2_m1 <- ggplot(m1_diag, aes(sample = .resid)) +
  stat_qq(alpha = 0.4) +
  stat_qq_line(color = "red") +
  labs(title = "M1: Normal Q-Q Plot",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()

#m2
# Add residuals and fitted values to the dataframe.
m2_diag <- augment(m2)

# Residuals vs Fitted for M2
p1_m2 <- ggplot(m2_diag, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(method = "loess", se = FALSE, color = "blue") +
  labs(title = "M2: Residuals vs Fitted",
       x = "Fitted Values",
```

```r
       y = "Residuals") +
  theme_minimal()

# Normal Q-Q Plot for M2
p2_m2 <- ggplot(m2_diag, aes(sample = .resid)) +
  stat_qq(alpha = 0.4) +
  stat_qq_line(color = "red") +
  labs(title = "M2: Normal Q-Q Plot",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()




# 4. Cross-Validation Functions

# 4. Cross-Validation Functions

calc_scores <- function(y, mu, sigma, alpha = 0.05, model_name = "Model") {
  # y     : vector of observed values
  # mu    : vector of predictive means (from predict(fit, newdata=test)$fit)
  # sigma : vector of predictive SDs. Combine residual error and mean uncertainty:
  #         sigma = sqrt(summary(fit)$sigma^2 +
  # predict(fit, newdata=test, se.fit=TRUE)$se.fit^2)
  # Returns a named list with RMSE, MAE, DS, IS

  # Implement RMSE, MAE, DS, and IS here

  #RMSE
  RMSE <- sqrt(mean((y - mu)^2))

  #MAE
  MAE <- mean(abs(y - mu))

  #DS
  DS <- mean(log(sigma^2) + ((y - mu)^2 / sigma^2))

  #IS
  lower <- mu - (qnorm(1-alpha/2) * sigma)
  upper <-mu + (qnorm(1-alpha/2) * sigma)
  IS <-  mean((upper - lower) +
               (2/alpha)* pmax(0,(lower - y)) +
               (2/alpha)*pmax(0,(y - upper)))

  # make a table with the results
  return (data.frame(Model = model_name, RMSE = RMSE, MAE = MAE, DS = DS, IS = IS))
}
```

```r
# 4. Cross-Validation Functions

calc_scores <- function(y, mu, sigma, alpha = 0.05, model_name = "Model") {
  # y     : vector of observed values
  # mu    : vector of predictive means (from predict(fit, newdata=test)$fit)
  # sigma : vector of predictive SDs. Combine residual error and mean uncertainty:
  #         sigma = sqrt(summary(fit)$sigma^2 +
  # predict(fit, newdata=test, se.fit=TRUE)$se.fit^2)
  # Returns a named list with RMSE, MAE, DS, IS

  # Implement RMSE, MAE, DS, and IS here

  #RMSE
  RMSE <- sqrt(mean((y - mu)^2))

  #MAE
  MAE <- mean(abs(y - mu))

  #DS
  DS <- mean(log(sigma^2) + ((y - mu)^2 / sigma^2))

  #IS
  lower <- mu - (qnorm(1-alpha/2) * sigma)
  upper <-mu + (qnorm(1-alpha/2) * sigma)
  IS <-  mean((upper - lower) +
               (2/alpha)* pmax(0,(lower - y)) +
               (2/alpha)*pmax(0,(y - upper)))

  # make a table with the results
  return (data.frame(Model = model_name, RMSE = RMSE, MAE = MAE, DS = DS, IS = IS))
}


# 5. Leave-One-Year-Out CV Loop
# Define models in a list ###change formulas.!!!
models_list <- list(
  m0 = count ~temp_mean + as.numeric(weekend) + as.numeric(month),
  m1 = count~ temp_mean + as.numeric(weekend) + as.numeric(month),
  m2 = count~ temp_mean + as.numeric(weekend) + as.numeric(month),
  m3 = count~ temp_mean + as.numeric(weekend) + as.numeric(month))

cycle_daily_df$date <- as.Date(cycle_daily_df$date)
cycle_daily_df$year <- year(cycle_daily_df$date)
years <- sort(unique(cycle_daily_df$year))
results_list <- list()

for (y in years) {
  # Split: Extract one year for testing
  train_data <- subset(cycle_daily_df, year != y)
  test_data  <- subset(cycle_daily_df, year == y)

  for (mod_name in names(models_list)) {
    # Fit the model on the remaining years
```

```r
    fit <- lm(models_list[[mod_name]], data = train_data)

    # Get predictive means (mu) and standard errors (se.fit)
    pred_obj <- predict(fit, newdata = test_data, se.fit = TRUE)
    mu <- pred_obj$fit

    # Calculate total predictive SD (sigma)
    # Combining residual variance and uncertainty in the mean
    res_std_error <- summary(fit)$sigma
    sigma <- sqrt(res_std_error^2 + pred_obj$se.fit^2)

    # Run your calc_scores function
    scores <- calc_scores(y = test_data$count, mu = mu, sigma = sigma, model_name = mod_name)

    # Store result with identifiers
    results_list[[paste(y, mod_name, sep="_")]] <- scores %>% mutate(Split_Year = y)

  }
}
#finding the average scores for each model and displaying
all_cv_results <- bind_rows(results_list)
table1_final <- all_cv_results %>%
  group_by(Model) %>%
  summarise(across(c(RMSE, MAE, DS, IS), mean))
table1_final




# 6. CV by Month
# ...
```